

Dokumentasi Sistem Deteksi Intrusi Berbasis YOLOv8

(Deteksi Penyusup & Senjata Api pada CCTV Area Terlarang)

1. Latar Belakang & Tujuan Proyek

1.1 Latar Belakang

Sistem keamanan CCTV konvensional umumnya hanya berfungsi sebagai perekam pasif — rekaman baru diperiksa setelah suatu insiden terjadi. Hal ini membuat respons terhadap ancaman (penyusup masuk ke area terlarang, atau munculnya senjata api) menjadi lambat karena bergantung pada pengawasan manual oleh petugas.

Proyek ini membangun **sistem deteksi intrusi otomatis berbasis computer vision**, yang mampu mengenali secara *real-time* dua kelas objek dari input CCTV:

- **intruder** — orang/penyusup yang masuk ke area yang dipantau
- **gun** — senjata api

Dengan deteksi otomatis, sistem dapat memberi peringatan dini (alert) tanpa harus menunggu pengawasan manusia secara terus-menerus.

1.2 Tujuan

1. Melatih model *object detection* yang akurat dan ringan untuk mendeteksi **intruder** dan **gun** dari citra CCTV.
2. Mengoptimalkan model agar dapat berjalan secara efisien di berbagai jenis perangkat keras (CPU biasa, CPU Intel dengan NPU, maupun GPU non-Intel), tanpa mengorbankan kemudahan deployment.
3. Membangun backend aplikasi web yang **otomatis mendeteksi hardware** yang tersedia dan memilih format model yang paling optimal secara otomatis (auto-detect & fallback), sehingga aplikasi tetap bisa berjalan di berbagai spesifikasi komputer tanpa perlu konfigurasi manual oleh pengguna.

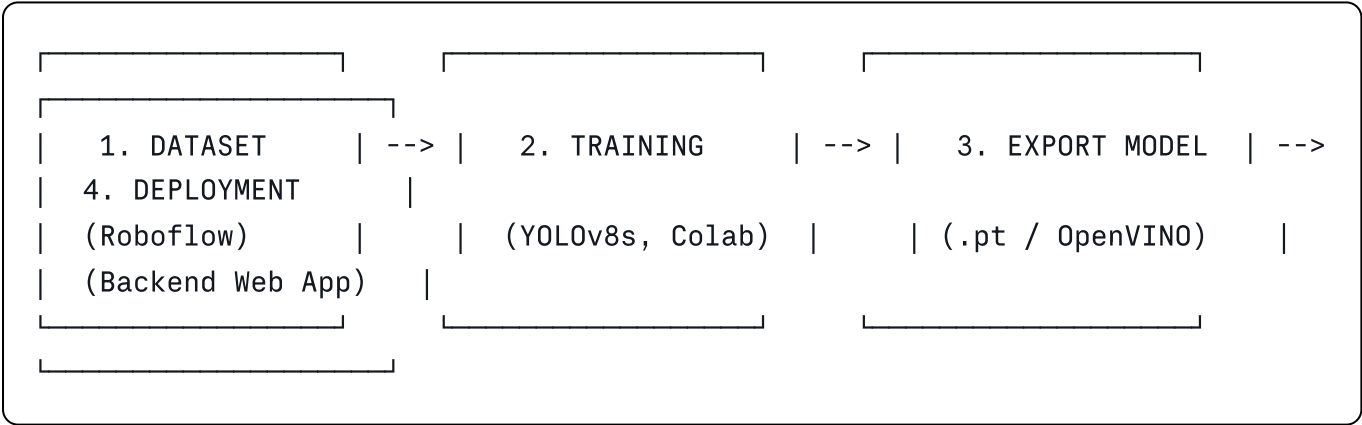
1.3 Ruang Lingkup

Proyek ini terbagi menjadi dua bagian besar yang saling terhubung:

Bagian	Lingkup	Sumber
A. Training & Optimasi Model	Melatih model YOLOv8 dari dataset CCTV, mengevaluasi, lalu mengekspor ke beberapa format (PyTorch, OpenVINO FP, OpenVINO INT8)	<code>Intrusion_Detection_YOLOv8_Training.ipynb</code> (Google Colab)
B. Deployment Backend (Web App)	Membangun logic backend yang otomatis memilih format model & device terbaik sesuai hardware yang dijalankan, dengan mekanisme fallback jika terjadi kegagalan	<code>Prompt_VibeCoding_AutoDetect_Hardware.md</code> (dieksekusi via AI coding tool di VS Code)

2. Arsitektur Pipeline End-to-End

Secara garis besar, alur sistem ini terdiri dari 4 tahap besar:



1. **Dataset** — data citra CCTV berlabel `gun` dan `intruder`, diambil dari Roboflow.
2. **Training** — model YOLOv8s dilatih di Google Colab menggunakan GPU (Tesla T4).
3. **Export Model** — model hasil training dikonversi ke beberapa format agar bisa dijalankan optimal di berbagai hardware.
4. **Deployment** — backend web app memuat model secara otomatis sesuai hardware yang mendeteksinya (auto-detect + fallback), lalu menjalankan inferensi real-time.

3. Bagian A — Training Model (Google Colab)

3.1 Dataset

Split	Jumlah Gambar	Fungsi
Train	2.551 gambar	Melatih bobot model
Valid	517 gambar	Evaluasi tiap epoch, dasar early stopping
Test	162 gambar	Evaluasi akhir performa model
Total	3.230 gambar	—

- Sumber dataset: **Roboflow**, diunggah dalam bentuk file ZIP lalu diekstrak ke `/content/dataset`.
- Konfigurasi path dataset diatur ulang di `data.yaml` agar sesuai struktur folder Colab (`train/images`, `valid/images`, `test/images`), termasuk daftar nama kelas (`gun`, `intruder`).

Catatan skala data: 3.230 gambar tergolong dataset skala kecil-menengah, cukup untuk baseline/tugas kuliah. Jika mAP masih rendah, ini wajar untuk ukuran dataset seperti ini — bisa didiskusikan sebagai *area pengembangan lanjutan* (misalnya penambahan data atau augmentasi lebih variatif).

3.2 Model & Metode: YOLOv8s

YOLO (You Only Look Once) adalah keluarga algoritma *object detection* berbasis *deep learning* satu-tahap (*single-stage detector*) yang memprediksi lokasi (bounding box) dan kelas objek dalam satu kali forward-pass jaringan saraf — inilah yang membuatnya jauh lebih cepat dibanding metode dua-tahap (seperti Faster R-CNN), sehingga cocok untuk kebutuhan **real-time**, seperti CCTV.

YOLOv8 adalah versi terbaru dari keluarga YOLO oleh Ultralytics, dengan arsitektur *backbone-neck-head* yang telah diperbarui untuk akurasi dan kecepatan lebih baik dibanding versi sebelumnya (YOLOv5).

Varian YOLOv8s (small) dipilih karena berada di titik keseimbangan (*sweet spot*) antara:

- **Akurasi** — lebih tinggi dibanding varian paling ringan (YOLOv8n / nano)
- **Kecepatan & ukuran** — masih cukup ringan untuk dilatih di GPU gratisan Google Colab (Tesla T4) dan nantinya dijalankan real-time di sisi deployment

3.3 Alur Training Detail

Tahap	Deskripsi
1. Upload & unzip dataset	Dataset ZIP dari Roboflow diunggah manual ke Colab, diekstrak ke <code>/content/dataset</code>
2. Perbaikan <code>data.yaml</code>	Path <code>train</code> , <code>val</code> , <code>test</code> disesuaikan ke struktur folder Colab
3. Cek jumlah data	Verifikasi jumlah gambar tiap split (train/val/test)
4. Instalasi & cek GPU	Install <code>ultralytics</code> , pastikan GPU (Tesla T4) terdeteksi via <code>torch.cuda.is_available()</code>
5. Training	<code>model.train()</code> dengan konfigurasi di bawah
6. Evaluasi	<code>model.val(split='test')</code> menghasilkan metrik mAP50, mAP50-95, precision, recall
7. Visualisasi hasil	Menampilkan <code>results.png</code> (grafik loss/metric per epoch) dan <code>confusion_matrix.png</code>
8. Inference sampel	Uji deteksi pada 1 gambar dari test set (<code>conf=0.4</code>)
9. Download <code>best.pt</code>	Bobot terbaik diunduh dari <code>/content/runs/intrusion_detection_v1/weights/best.pt</code>

3.4 Konfigurasi Training (Hyperparameter)

```
python

model = YOLO('yolov8s.pt') # pretrained weights sebagai starting point (transfer learning)

results = model.train(
    data='/content/dataset/data.yaml',
    epochs=100,             # maksimum 100 epoch (putaran penuh data training)
    imgsz=640,              # ukuran gambar input 640x640 piksel
    batch=16,               # 16 gambar per batch
    patience=20,            # early stopping: berhenti jika 20 epoch tanpa peningkatan
    optimizer='AdamW',      # optimizer adaptif, stabil untuk fine-tuning
    # + augmentasi data: rotasi, scaling, flip horizontal, mosaic
    # + scheduler: Cosine Learning Rate
)
```

Penjelasan strategi training:

- **Transfer learning** — model tidak dilatih dari nol, melainkan mulai dari bobot `yolov8s.pt` yang sudah dilatih di dataset besar (COCO), lalu di-*fine-tune* ke dataset spesifik (`gun`, `intruder`). Ini mempercepat konvergensi dan meningkatkan akurasi meski dataset relatif kecil.
- **Augmentasi data** (rotasi, scaling, flip horizontal, mosaic) — bertujuan agar model tidak hanya "menghafal" gambar training, melainkan belajar mengenali objek dalam berbagai posisi, skala, dan pencahayaan, sehingga lebih tahan (*robust*) terhadap kondisi CCTV nyata yang bervariasi.
- **AdamW + Cosine Learning Rate Scheduler** — kombinasi ini membantu proses pembelajaran bobot model berjalan lebih stabil dan halus, mengurangi risiko learning rate yang terlalu besar/kecil di fase akhir training.
- **Early stopping** (`patience=20`) — mencegah *overfitting* dan menghemat waktu komputasi; training otomatis dihentikan bila tidak ada perbaikan performa selama 20 epoch berturut-turut, walau batas maksimum diset 100 epoch.

3.5 Evaluasi Model — Hasil Aktual

Setelah training, model dievaluasi pada **test set** (data yang belum pernah dilihat model) menggunakan metrik standar object detection:

Metrik	Arti
mAP50	Rata-rata presisi deteksi saat IoU (overlap bounding box) $\geq 50\%$ — metrik umum untuk menilai "apakah objek terdeteksi dengan cukup akurat"
mAP50-95	Rata-rata mAP dihitung pada rentang IoU 50%–95%, metrik yang lebih ketat/komprehensif
Precision	Dari semua deteksi yang dibuat model, berapa persen yang benar (menghindari false positive)
Recall	Dari semua objek yang sebenarnya ada, berapa persen yang berhasil terdeteksi (menghindari false negative)

Hasil training (real, diambil dari output notebook):

- Training berhenti otomatis di **epoch ke-72 dari 100** (early stopping aktif — tidak ada peningkatan performa selama 20 epoch berturut-turut). Bobot terbaik tercatat pada **epoch 52**.
- Waktu training total: **$\pm 1,06$ jam** di GPU Tesla T4.

Hasil evaluasi pada test set (162 gambar, model `best.pt`):

Metrik	Nilai
mAP50	0.8931 (89,3%)
mAP50-95	0.5117 (51,2%)
Precision	0.8553 (85,5%)
Recall	0.8865 (88,7%)

Breakdown per kelas (dari hasil validasi saat training, val set 517 gambar):

Kelas	Jumlah Instance	Precision	Recall	mAP50	mAP50-95
<code>intruder</code>	752	0.831	0.801	0.846	0.459
<code>gun</code>	4	1.000	0.000	0.259	0.131
Rata-rata (all)	756	0.915	0.400	0.553	0.295

⚠ **Temuan penting:** kelas `intruder` terdeteksi dengan sangat baik (mAP50 $\approx$ 85%), tetapi kelas `gun` performanya jauh lebih rendah (mAP50 hanya $\approx$ 26%). Penyebab utamanya terlihat jelas dari datanya: instance `gun` di val set **hanya berjumlah 4**, dibanding 752 instance `intruder` — dataset sangat tidak seimbang (*class imbalance*). Model jadi kurang punya cukup contoh untuk benar-benar belajar mengenali senjata api secara andal. Ini **wajib disebutkan sebagai limitasi utama** di laporan, dan idealnya jadi prioritas pengembangan lanjutan (lihat Bab 6) — misalnya menambah data gambar `gun` secara signifikan sebelum model ini dipakai untuk skenario nyata yang melibatkan deteksi senjata.

3.6 Ekspor Model ke Multi-Format

Setelah training selesai, model `best.pt` diekspor ke **3 varian format** agar bisa dipilih sesuai kebutuhan hardware saat deployment:

a) PyTorch (`best.pt`)

Format asli hasil training. Fleksibel, dijalankan menggunakan library `ultralytics`/`torch`, cocok untuk perangkat dengan GPU NVIDIA (CUDA) atau CPU umum (termasuk non-Intel/AMD).

b) OpenVINO FP (`best_openvino_model/`)

```
python
model.export(format='openvino', imgsz=640, half=False)
```

- **OpenVINO** adalah toolkit optimasi inferensi buatan Intel, dirancang agar model deep learning berjalan lebih cepat & efisien di hardware Intel (CPU, GPU integrated, maupun NPU).
- `half=False` berarti tetap presisi penuh (FP32); bisa diubah ke `half=True` untuk FP16 (lebih ringan, sedikit menurunkan akurasi).
- Hasil ekspor berupa 2 file: `best.xml` (struktur/arsitektur model) dan `best.bin` (bobot model).

c) OpenVINO INT8 (`best_int8_openvino_model/`)

```
python

model.export(
    format='openvino',
    imgsz=640,
    int8=True,          # aktifkan quantization
    data='/content/dataset/data.yaml', # dibutuhkan untuk kalibrasi
    dynamic=False,      # wajib static shape, terutama untuk NPU
    half=False
)
```

- Menggunakan **NNCF (Neural Network Compression Framework)** untuk melakukan *quantization* dari FP32 → INT8.
- **Quantization** mengubah presisi angka bobot model dari 32-bit menjadi 8-bit, sehingga ukuran model jauh lebih kecil dan inferensi jauh lebih cepat — dengan trade-off potensi sedikit penurunan akurasi.
- `dynamic=False` **wajib** karena NPU (Neural Processing Unit) pada CPU Intel generasi baru (Intel Core Ultra) hanya mendukung *input shape* statis (ukuran tetap), tidak dinamis.

3.7 Perbandingan & Benchmark 3 Model — Hasil Aktual

Notebook ini melakukan **head-to-head benchmark** ketiga model (`.pt`, OpenVINO FP, OpenVINO INT8) pada test set yang sama, lalu pada data baru (foto CCTV di luar dataset training) untuk menguji generalisasi.

A. Akurasi pada test set (162 gambar):

Metrik	PyTorch (.pt)	OpenVINO FP	OpenVINO INT8
mAP50	0.8931	0.8833	0.8819
mAP50-95	0.5117	0.4778	0.4754
Precision	0.8553	0.8855	0.8902
Recall	0.8865	0.8417	0.8333

B. Kecepatan & akurasi pada data baru (1 foto CCTV di luar dataset, jalur Sudirman jam pulang kantor — 7 objek `intruder`):

Model	Total Waktu	Rata ² Waktu/Gambar	FPS (estimasi)	Objek Terdeteksi	Rata ² Confidence
PyTorch (.pt)	0.139 s	0.139 s	7.21	7	0.7569
OpenVINO FP	0.536 s	0.536 s	1.87	7	0.7597
OpenVINO INT8	0.665 s	0.665 s	1.50	7	0.7679

⚠ **Temuan penting kedua:** pada benchmark ini, justru `.pt` (PyTorch) yang paling cepat (7,21 FPS), bukan OpenVINO — kebalikan dari ekspektasi umum (OpenVINO biasanya lebih cepat *di hardware Intel yang sesuai*). Ini terjadi karena environment Colab yang dipakai untuk benchmark menggunakan GPU CUDA (Tesla T4) untuk `.pt`, sementara OpenVINO di notebook ini berjalan di mode CPU ("Using OpenVINO LATENCY mode... on CPU"), bukan di hardware Intel NPU/GPU yang sebenarnya. Artinya:

- Angka benchmark ini **valid untuk kondisi Colab** (GPU NVIDIA vs OpenVINO-CPU), tapi **belum mencerminkan performa asli OpenVINO di laptop/PC Intel dengan NPU/GPU terintegrasi** — yang menjadi target sebenarnya dari sistem *auto-detect hardware* di Bagian B.
- Untuk laporan yang akurat, benchmark OpenVINO **sebaiknya diulang langsung di perangkat Intel fisik** (idealnya yang punya NPU, seperti Intel Core Ultra) agar klaim "OpenVINO lebih optimal di hardware Intel" punya bukti kuantitatif yang tepat konteks — bukan dibandingkan dengan GPU NVIDIA di Colab.
- Meski lebih lambat di benchmark ini, OpenVINO INT8 menunjukkan **confidence rata-rata tertinggi** (0.7679) dan **precision tertinggi** (0.8902) — jadi ada trade-off: `.pt`+GPU unggul di kecepatan mentah, OpenVINO INT8 sedikit lebih presisi dan hemat ukuran (11,2 MB, dari log ekspor).

Hasil lengkap juga tersimpan sebagai `perbandingan_model_data_baru.csv` di notebook.

Tujuan bagian ini: memberi bukti kuantitatif bahwa strategi *auto-detect hardware* pada bagian deployment (Bagian B) beralasan — karena ketiga format model punya karakteristik kecepatan vs akurasi yang berbeda tergantung hardware yang menjalankannya, meski pengujian yang representatif untuk hardware Intel nyata masih perlu dilakukan terpisah dari Colab.

3.8 Output Akhir dari Tahap Training

File/Folder	Format	Peruntukan
best.pt	PyTorch	Non-Intel (AMD) / fallback universal
best_openvino_model/ (.xml + .bin)	OpenVINO FP32	Intel CPU/GPU/NPU, presisi penuh
best_int8_openvino_model/ (.xml + .bin)	OpenVINO INT8	Intel CPU/GPU/NPU, versi terkuantisasi (lebih cepat & ringan)

4. Bagian B — Deployment: Backend Auto-Detect Hardware

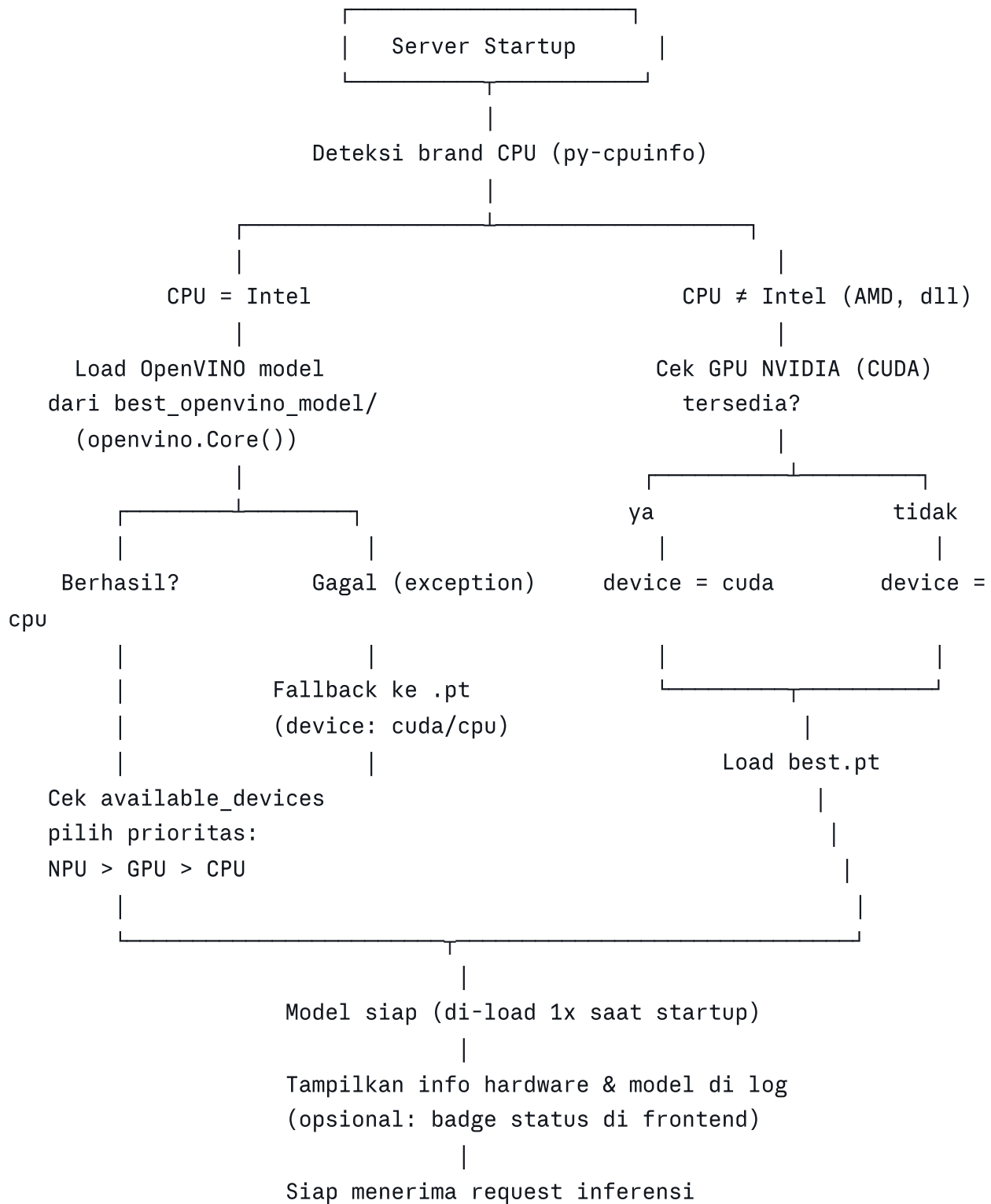
Bagian ini adalah instruksi (prompt) yang dieksekusi lewat AI coding tool di VS Code untuk membangun **backend aplikasi web** yang memuat model hasil training di atas secara cerdas, menyesuaikan hardware yang menjalankannya.

4.1 Masalah yang Diselesaikan

Tanpa mekanisme ini, developer harus **manual** memilih format model sesuai spesifikasi komputer target — merepotkan dan rawan error (misalnya OpenVINO gagal di-load di CPU non-Intel, atau model `.pt` berjalan lambat di CPU Intel yang sebenarnya punya NPU/GPU terintegrasi yang lebih cepat).

Solusinya: backend **mendeteksi sendiri** hardware yang tersedia saat startup, lalu memilih format & device model yang paling optimal secara otomatis — dengan mekanisme *fallback* berlapis jika terjadi kegagalan.

4.2 Logic Auto-Detect (Flow Lengkap)



4.3 Detail Logic per Skenario

Skenario Hardware	Format Model	Device	Fallback
Intel tanpa NPU (mis. i3 gen 10)	OpenVINO	CPU (satu-satunya device tersedia)	Jika load OpenVINO gagal → <code>.pt</code>
Intel dengan NPU (mis. Core Ultra)	OpenVINO	NPU (prioritas tertinggi) → GPU → CPU	Jika load OpenVINO gagal → <code>.pt</code>
Non-Intel (mis. AMD)	PyTorch (<code>.pt</code>)	<code>cuda</code> jika GPU NVIDIA tersedia, else <code>cpu</code>	— (langsung <code>.pt</code> , tidak mencoba OpenVINO)

Poin penting desain:

- 1. **Deteksi brand CPU** dilakukan sekali di awal (startup) memakai `py-cpuinfo`, bukan di setiap request — karena informasi hardware tidak berubah selama aplikasi berjalan.
- 2. **Model hanya di-load sekali** saat server pertama kali start dan disimpan di memori (bukan re-load setiap ada request masuk) — ini krusial untuk performa, karena loading model (terutama OpenVINO) memakan waktu cukup signifikan.
- 3. **Prioritas device OpenVINO:** NPU > GPU > CPU — dicek lewat `openvino.Core().available_devices`, karena NPU (jika ada) biasanya paling efisien untuk inferensi AI ringan dengan konsumsi daya rendah, cocok untuk aplikasi yang berjalan terus-menerus (24/7) seperti monitoring CCTV.
- 4. **Fallback berlapis:** jika OpenVINO gagal dimuat (misalnya karena file model rusak/tidak lengkap, versi OpenVINO tidak kompatibel, dll), sistem otomatis beralih ke `.pt` tanpa membuat aplikasi crash.
- 5. **Fail-safe terakhir:** jika **kedua** format model (OpenVINO maupun `.pt`) gagal dimuat, sistem harus menampilkan pesan error yang jelas ke user, bukan crash tanpa keterangan.

4.4 Struktur Proyek

```
project/
├─ best.pt                # model PyTorch – untuk non-Intel /
fallback
├─ best_openvino_model/  # model OpenVINO FP – untuk Intel
│   └─ best.xml
│   └─ best.bin
├─ model_loader.py        # modul khusus: logic deteksi hardware &
load model
├─ app.py                 # aplikasi utama backend, import dari
model_loader.py
```

Catatan pengembangan: Struktur di atas belum menyertakan

`best_int8_openvino_model/` (hasil quantization dari notebook). Jika Anda ingin memanfaatkan varian INT8 yang lebih cepat (hasil benchmark di Bagian A.3.7), struktur dan logic `model_loader.py` perlu ditambah satu cabang lagi — misalnya menjadikan INT8 sebagai pilihan utama untuk device NPU (karena NPU mewajibkan static shape yang sudah dipenuhi opsi `dynamic=False` saat ekspor), dengan OpenVINO FP sebagai fallback kedua sebelum jatuh ke `.pt`.

4.5 Dependencies Backend

```
py-cpuinfo      # deteksi brand & informasi CPU
opencvino       # runtime inferensi teroptimasi untuk hardware Intel
ultralytics     # library YOLOv8, dipakai untuk load & inference kedua format
torch           # dibutuhkan untuk menjalankan model .pt
```

4.6 Penanganan Error

Seluruh proses deteksi hardware dan loading model dibungkus `try-except` berlapis:

```
try:
    deteksi CPU
    if Intel:
        try:
            load OpenVINO, pilih device (NPU>GPU>CPU)
        except:
            load .pt sebagai fallback
    else:
        load .pt langsung (device cuda/cpu)
except (kedua model gagal):
    tampilkan pesan error jelas ke user:
    "Model tidak dapat dimuat. Pastikan file best.pt atau
    best_openvino_model tersedia di folder project."
```

Prinsip desainnya: **aplikasi tidak boleh crash tanpa keterangan** — setiap kegagalan harus tertangani dan menghasilkan pesan yang informatif, baik di log server maupun (idealnya) ke pengguna.

4.7 Observability (Logging & Frontend Badge)

- **Log server saat startup, contoh:**

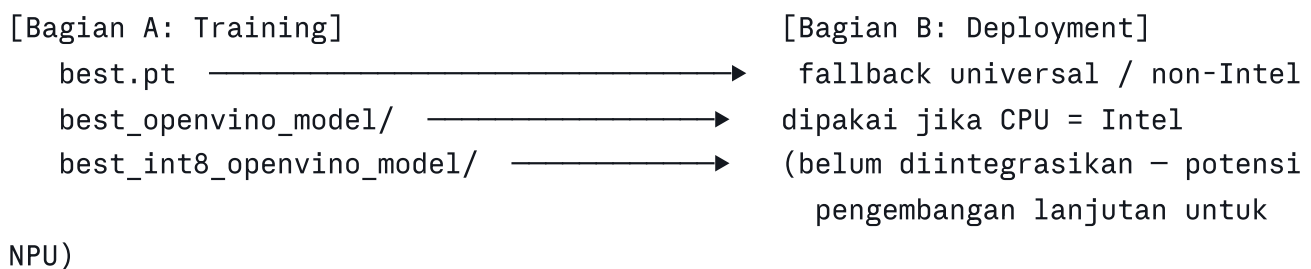
```
CPU terdeteksi: Intel(R) Core(TM) i3-10100
Brand: Intel
OpenVINO device tersedia: ['CPU']
Model digunakan: OpenVINO (best_openvino_model), device: CPU
```

- **(Opsional) Badge di frontend**, misalnya di pojok halaman: "*Running on: Intel CPU (OpenVINO)*" atau "*Running on: AMD (PyTorch)*" — memberi transparansi ke pengguna mengenai mode inferensi yang sedang aktif tanpa harus membuka log server.

4.8 Integrasi dengan Fitur Tracking

Jika aplikasi juga menggunakan fitur *object tracking* (ByteTrack/BoT-SORT) untuk melacak pergerakan objek antar-frame video (bukan sekadar deteksi per-frame), tracker ini harus tetap konsisten digunakan **terlepas dari format model mana yang aktif** (OpenVINO maupun `.pt`) — karena tracker beroperasi di atas hasil deteksi (bounding box), bukan bergantung pada format model itu sendiri.

5. Ringkasan Keterkaitan Bagian A & B



Ketiga model hasil ekspor dari notebook training menjadi **input langsung** bagi logic `model_loader.py` di sisi backend. Dengan kata lain, Bagian A menyiapkan "amunisi" model dalam berbagai format, dan Bagian B menyiapkan "logic pemilihan otomatis" agar amunisi tersebut dipakai sesuai kondisi medan (hardware pengguna).

6. Potensi Pengembangan Lanjutan

Berdasarkan hasil aktual di atas, urutan prioritas pengembangan lanjutan:

1. **[Prioritas tertinggi] Tambah data kelas `gun` secara signifikan** — dengan hanya 3-4 instance di data validasi/test, model praktis belum benar-benar belajar mengenali senjata api (mAP50 kelas ini hanya $\approx 26\%$, recall bahkan 0%). Ini adalah gap paling kritis karena deteksi senjata adalah salah satu tujuan utama sistem.
2. **Ulangi benchmark OpenVINO di hardware Intel fisik** (idealnya Intel Core Ultra dengan NPU) — benchmark saat ini di Colab membandingkan `.pt`+GPU NVIDIA vs OpenVINO+CPU, sehingga belum representatif untuk skenario deployment sebenarnya di Bagian B.
3. **Integrasi model INT8 ke dalam `model_loader.py`** sebagai opsi tambahan untuk device NPU (saat ini prompt deployment baru mencakup `.pt` dan OpenVINO FP, belum INT8).
4. **Penambahan data training secara umum** — 3.230 gambar tergolong skala kecil-menengah; variasi tambahan (sudut CCTV berbeda, kondisi malam hari, jarak objek bervariasi) berpotensi meningkatkan mAP50-95 yang saat ini masih di kisaran 51% (cukup rendah dibanding mAP50 89% , tanda bounding box belum terlalu presisi).

5. **Threshold confidence adaptif** — saat ini inferensi memakai `conf=0.4` tetap; bisa dieksplorasi apakah nilai ini optimal, terutama karena kelas `gun` butuh sensitivitas berbeda dari `intruder`.
 6. **Alerting/notifikasi** — sistem saat ini fokus pada deteksi; penambahan mekanisme notifikasi (misalnya suara alarm, push notification) akan melengkapi sistem menjadi solusi keamanan yang lebih utuh.
-

Dokumen ini disusun berdasarkan isi `Intrusion_Detection_YOLOv8_Training.ipynb` (termasuk seluruh output eksekusi yang tersimpan di dalamnya) dan `Prompt_VibeCoding_AutoDetect_Hardware.md`. Seluruh angka metrik pada Bab 3 adalah hasil aktual dari run notebook, bukan estimasi.